

SmartSAGE: Training Large-scale Graph Neural Networks using In-Storage Processing Architectures

Yunjae Lee
School of Electrical Engineering
KAIST
yunjae408@kaist.ac.kr

Jinha Chung
School of Electrical Engineering
KAIST
jinha.chung@kaist.ac.kr

Minsoo Rhu
School of Electrical Engineering
KAIST
mrhu@kaist.ac.kr

ABSTRACT

Graph neural networks (GNNs) can extract features by learning both the representation of each objects (i.e., graph nodes) and the relationship across different objects (i.e., the edges that connect nodes), achieving state-of-the-art performance in various graph-based tasks. Despite its strengths, utilizing these algorithms in a production environment faces several challenges as the number of graph nodes and edges amount to several billions to hundreds of billions scale, requiring substantial storage space for training. Unfortunately, state-of-the-art ML frameworks employ an in-memory processing model which significantly hampers the productivity of ML practitioners as it mandates the overall working set to fit within DRAM capacity. In this work, we first conduct a detailed characterization on a state-of-the-art, large-scale GNN training algorithm, GraphSAGE. Based on the characterization, we then explore the feasibility of utilizing capacity-optimized NVMe SSDs for storing memory-hungry GNN data, which enables large-scale GNN training beyond the limits of main memory size. Given the large performance gap between DRAM and SSD, however, blindly utilizing SSDs as a direct substitute for DRAM leads to significant performance loss. We therefore develop SmartSAGE, our software/hardware co-design based on an in-storage processing (ISP) architecture. Our work demonstrates that an ISP based large-scale GNN training system can achieve both high capacity storage and high performance, opening up opportunities for ML practitioners to train large GNN datasets without being hampered by the physical limitations of main memory size.

CCS CONCEPTS

• **Computer systems organization** → **Neural networks**; • **Hardware** → **External storage**; • **Computing methodologies** → **Machine learning**; **Artificial intelligence**.

KEYWORDS

Graph neural network, computational storage device, near data processing, solid state drives (SSD)

ACM Reference Format:

Yunjae Lee, Jinha Chung, and Minsoo Rhu. 2022. SmartSAGE: Training Large-scale Graph Neural Networks using In-Storage Processing Architectures. In *The 49th Annual International Symposium on Computer Architecture (ISCA '22)*, June 18–22, 2022, New York, NY, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3470496.3527391>

1 INTRODUCTION

Deep neural network (DNN) based machine learning (ML) algorithms are providing super-human performance in areas of image classification, natural language processing, speech recognition, and others. However, such DNN based ML design paradigms primarily targeted Euclidean data (e.g., image, text, and audio), having limited adoption in domains where *non-Euclidean* data structures such as *graphs* are utilized. Recently, *graph neural networks* (GNNs) have emerged as a powerful tool in application domains that target arbitrarily structured graph inputs, where feature vectors are associated with graph nodes and edges. GNNs have found significant success in the areas of e-commerce and advertisement where the graph nodes and edges represent objects and their relationships, providing unparalleled performance in traditional graph analytics workloads. For instance, Pinterest’s PinSAGE [85] or Alibaba’s AliGraph [90] leverages GNNs to analyze and extract high quality features from graphs with billions of user/item feature embeddings, achieving state-of-the-art performance.

With the proliferation of GNNs, we are witnessing a large number of ML frameworks tailored for graph learning being developed, examples of which include Deep Graph Library (DGL) [78] and PyTorch Geometric (PyG) [21]. Unlike conventional DNNs (e.g., convolutional, recurrent, or fully-connected layers) which exhibit a highly regular and dense dataflow, GNN training inherently contains a *hybrid* mix of both sparse and dense dataflows (Figure 1). More concretely, the frontend stages of GNN training which conduct “input data preparation” (e.g., graph neighbor sampling, feature table lookup) follow the typical graph analytics’ sparse and irregular memory access characteristics. In contrast, the backend stages which conduct “graph learning” (e.g., graph convolutions) employ well-structured, dense DNN algorithms using multi-layer perceptron (MLP) layers. Therefore, DGL and PyG provides dedicated user-level APIs tailored to the sparse dataflow of input data preparation stages (i.e., fine-grained feature gather-scatter for graph neighbor sampling and feature aggregation) which eases the programming of irregular and sparse frontend stages of GNN training.

While DGL and PyG help improve the programmability of GNNs, both frameworks employ the *in-memory* processing model [21, 78] (i.e., the target graph nodes/edges and its feature embedding tables must be stored inside main memory) which limits ML practitioners from scaling “up” the graph dataset, significantly hampering user

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ISCA '22, June 18–22, 2022, New York, NY, USA

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-8610-4/22/06...\$15.00

<https://doi.org/10.1145/3470496.3527391>

productivity. General trend in the GNN research space has been to increase the number of graph nodes as well as the number of edges (i.e., larger and more complex graphs) while employing more sophisticated DNNs for feature extraction (i.e., convolutions [41] to attentions [75]). An important reason why such scaled-up GNNs became prevalent is because it helps improve GNN’s algorithmic performance, similar to how ML algorithms targeting Euclidean data show higher performance with larger and deeper DNNs. Unfortunately, current ML frameworks’ in-memory processing model forces ML practitioners to tune the GNN training algorithm to fit within the several tens to hundreds of GBs of CPU memory, preventing developers from scaling up the graph network structure as well as its feature vector size. To this end, this paper explores the feasibility of utilizing NAND flash-based non-volatile memory (NVM) solutions to address the memory “capacity” bottlenecks of large-scale GNN training. Our proposal encompasses innovations at both the software and hardware stack, as detailed below.

Software. Given the wide performance gap between DRAM and NVMe SSDs, the central research challenge lies in how system architects should go about harmoniously architecting the memory-storage hierarchy that is most appropriate for GNN training’s algorithmic properties. Consequently, we start by conducting a characterization on a state-of-the-art large-scale GNN training algorithm, GraphSAGE [26]. Our characterization reveals that GNN training’s frontend input data preparation stage (Figure 1) is the most memory capacity intensive and becomes a prime candidate to be offloaded to SSDs. We therefore establish a baseline SSD-centric training system which stores the memory-hungry data structures (i.e., graph nodes/edges) inside SSDs. These data structures are mapped to user-space memory address via memory-mapped (mmap) file I/O, which allows the most recently accessed pages to be buffered inside the OS managed page cache (i.e., stored in DRAM), potentially narrowing the large performance gap between DRAM and SSDs.

Unfortunately, our baseline SSD-centric training system is shown to benefit little from the locality-optimized OS page cache, incurring high performance loss when offloaded to SSDs with an average 9.8× slowdown vs. an oracular, in-memory processing based system (i.e., all graph datasets can be stored in DRAM). Careful examination of the bottlenecks caused by the SSD-offloaded data preparation reveals the following **key observation**: because data preparation exhibits fine-grained irregular parallelism, it becomes challenging for the page cache to reap locality benefits, only to add several tens of microseconds of latency in traversing through the system software stack to maintain the page cache, rendering the frontend data preparation stage to be highly latency limited. As such, our proposal designs the software architecture to be optimized for latency, rather than locality. More concretely, we restructure ML framework’s software runtime system as well as the host driver stack to *bypass* the OS system software layers and directly access the SSD without page caching. Such design point obviates the latency overheads in maintaining the page cache to buffer recently accessed data, significantly reducing the time taken to fetch graph datasets from the SSD. We demonstrate that optimizing the DRAM↔SSD data movements for latency leads to significant speedup on end-to-end GNN training time with an average 2.5× vs. the baseline mmap-based SSD, closing the performance gap between DRAM vs. SSDs to “only” an average 3.8×.

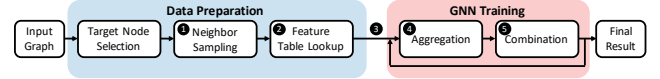


Figure 1: High-level overview of a large-scale GNN training pipeline.

Hardware. To further close DRAM-vs-SSD’s remaining performance gap, we also innovate at the hardware architecture level driven by both recent technological trends and GNN algorithm awareness. Recent trends point to the emergence of SSD storage devices that natively support *in-storage processing* (ISP) capabilities (e.g., Samsung-Xilinx’s SmartSSD [66], NGD system’s Newport [15, 71], Eideticom’s NoLoad CSP [19]) that offer fast communication between the flash devices and the ISP units. The key objective of our proposal is to synergistically combine ISP capabilities of these emerging computational storage devices (CSDs) with our latency-optimized software runtime system and host driver stack to achieve “DRAM-level” effective throughput, as provided with the in-memory processing baseline ML frameworks.

To this end, we develop SmartSAGE, an ISP based GNN training system that intelligently offloads the data intensive stages to the ISP unit, closely coupled inside the SSD. The key research problem naturally lies in identifying which part of the GNN training algorithm is most appropriate to be handled by our ISP architecture. Our characterization reveals that the nature of GNN training’s data preparation is effectively a *reduction* operation as it seeks to extract out multiple “subgraphs” from a much larger input graph. Rather than having large, coarse-grained chunks of the input graph be transferred from SSD to DRAM for subgraph generation by the host CPU, SmartSAGE offloads the data intensive steps of data preparation (more specifically, the *neighbor sampling* step) to the ISP units. This allows SmartSAGE to only transfer the subgraphs from SSD to DRAM, preprocessed by the ISP unit, significantly reducing the data movements between SSD→DRAM by an average 20×. Putting everything together, SmartSAGE holistically combines our latency-optimized software system with an ISP accelerated hardware architecture, achieving substantial speedup on end-to-end GNN training time with an average 3.5× (max 5.0×) vs. the baseline SSD-centric system. To summarize our key contributions:

- We conduct a detailed characterization on the data-intensive frontend data preparation stage of large-scale GNN training, root-causing several limitations of conventional in-memory processing GNN training systems.
- Driven by our characterization, we motivate and explore the viability of exploiting NVMe SSDs as a direct substitute for capacity limited DRAM based training systems.
- To bridge the wide performance gap between DRAM vs. SSD, we co-design the software/hardware of large-scale GNN training, presenting our latency-optimized software coupled with an in-store processing architecture, achieving superior performance than baseline SSD-centric systems.

2 BACKGROUND

2.1 Graph Neural Networks

GNNs are a variant of DNNs that operate over graph data structures. A unique property of GNNs is that they try to extract features by learning both the representation of each objects (i.e., graph nodes)

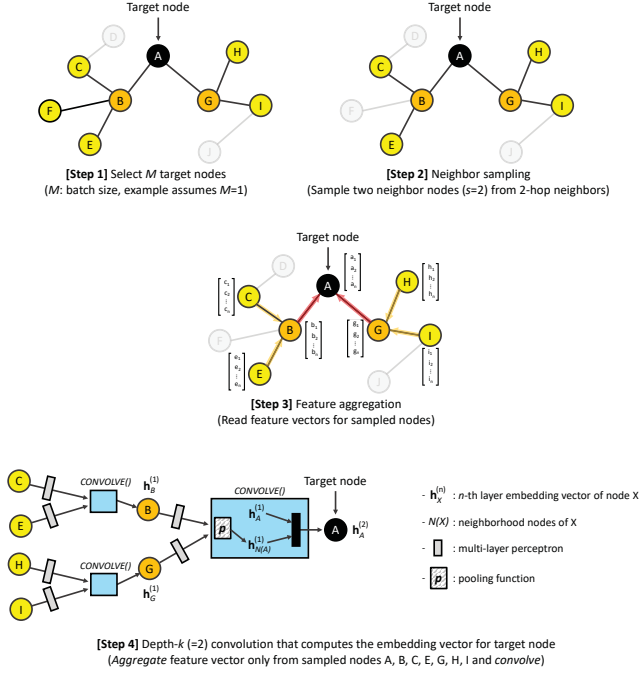


Figure 2: Illustration of the GraphSAGE sample and aggregate operator generating a single target node's embedding vector through a depth-2 convolution (considering upto two-hop ($k=2$) neighboring nodes). Training GNNs with a mini-batch of M involves selecting M target nodes (i.e., $M=1$ in this figure), each going through the sample and aggregation process. Example assumes two neighboring nodes are sampled per each target node (i.e., $s=2$). For instance, among the three neighbor nodes of node B (nodes $C/F/E$ in step 1), we only include nodes C and E (step 2) as part of the subgraph. In this work, we focus on addressing the bottlenecks incurred in step 1 and 2.

as well as the relationship across different objects (i.e., the edges that connect nodes). Consider the task of recommending a video/movie clip in an online video streaming service (e.g., YouTube, Netflix). A purely content based approach (e.g., recurrent neural networks [6]) would represent each object under consideration (i.e., the movie) based on the features derived from the target object (e.g., the genre of the movie). However, there can be valuable information existent across two (and potentially many) distinct objects, as there could be common properties that these objects share. By modeling such relationship as graph edges and each individual objects as graph nodes, a GNN can *learn* to extract meaningful feature representations not only from the object itself but also from the edges that are associated with it.

Consequently, GNN-based ML applications are achieving state-of-the-art performance on a wide range of graph-based algorithms, which include node/edge prediction [17, 22, 41], graph clustering [86], recommendation models [14], and others. Graph convolutional neural networks for instance aggregate target nodes' features and context information from their k -hop neighborhood nodes, similar to how conventional convolutional layers of a DNN application extracts features from a given pixel's neighborhood pixels. By stacking multiple of such convolution operations in sequence (e.g., two convolution layers consider up to two-hop ($k=2$) neighborhoods), the coverage of feature learning could exponentially propagate far reaches of the graph.

Algorithm 1 Neighbor sampling for “subgraph” generation

```

1: Set of target nodes  $M$ ; neighborhood  $N$ ; sampling size  $s$ ; Sampled set of nodes  $S$ 
2:
3: /* Sampling neighborhood nodes of target nodes */
4:  $S \leftarrow \emptyset$ 
5: for  $u \in M$  do
6:   for  $i \leftarrow 0$  to  $s$  do
7:     /* Randomly select its neighborhood nodes */
8:      $v \leftarrow \text{RandomSelect}(N(u))$ 
9:      $S \leftarrow S \cup v$ 
10:  end for
11: end for

```

2.2 “Sample and Aggregate” for Large-scale GNN Training

While GNNs have established a new standard in a variety of applications within the academic community, they have only recently started being deployed in practical, real-world problems [85, 90]. A key challenge that production environments face is that the number of graph nodes and edges amount to several billions to hundreds of billions scale, requiring substantial compute and memory for training and deployment. Training early GNN models [41] required operating on the *full* graph Laplacian during training, so the *entire* graph data and intermediate states of all graph nodes must be stored in memory. Such high compute and memory requirements can only be met when the target graph data structure is at a small scale.

To address the practical needs of scaling GNN training and deployment to massive, “web-scale” graph data, the seminal work on GraphSAGE (short for graph *sampling* and *aggregation* [26, 85]) proposed a highly scalable GNN training framework enabling large-scale graph learning (Figure 1). Rather than targeting the entire graph nodes and all the accompanying neighbor nodes for training, GraphSAGE first chooses a fixed number of M target nodes (M being equivalent to the training mini-batch size, typically in the range of several thousands), which is much smaller than the number of entire nodes, e.g., $M=1$ in step 1 of Figure 2. For each target node, GraphSAGE then “samples” s neighborhood nodes around each target node (Algorithm 1, step 2 of Figure 2), and only those sampled nodes near the target nodes will later be targeted for GNN training (i.e., the aggregation stage followed by combination in Figure 1). In effect, GraphSAGE dynamically constructs a *subgraph* (generated over a mini-batch of M target nodes, each target node containing s sampled nodes among its neighbors) and iteratively conducts convolutions only around the subgraph (step 3 and 4 in Figure 2). This allows the total number of target nodes for mini-batch training as well as the number of neighboring nodes for a given target node all be hyperparameters of the training algorithm, which helps drastically reduce the *active* compute and memory requirements of GNN training. As a result, GNNs are gradually seeing real-world adoption in consumer facing products, a well-known example being the usage at Pinterest (through a GNN called PinSAGE [85]) to generate embedding feature vectors for images/etc. Given their wide adoption and general applicability in enabling large-scale GNN training,

2.3 System Architecture for GNN Training

A GNN training algorithm's graph dataset consists of several key data structures including the graph neighbor edge list array that

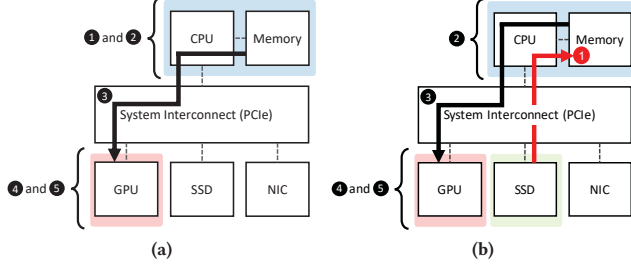


Figure 3: (a) The hybrid CPU-GPU based GNN training system assuming an in-memory processing model. (b) The baseline SSD-centric training system utilizes NVMe SSDs to store the memory-hungry neighbor edge list array which forces the neighbor sampling stage (step ①) to be conducted over the SSD, rather than DRAM, causing performance overheads. Note that step ① to ⑤ in this figure each matches the corresponding steps ①–⑤ in Figure 1.

encapsulates the structure of the graph using its adjacency matrix, and the feature table which abstracts each node’s unique property as a feature vector. In real-world graphs, the number of graph edges outweighs the number of graph nodes, so the overall memory consumption is generally dominated by the neighbor edge list array (discussed further in Section 4.2/Figure 10) rather than the feature table, the aggregate size of which can amount to several hundreds to thousands of GBs for large-scale graphs. Such constraint poses several key challenges in designing the overall system architecture for large-scale GNN training.

Consequently, state-of-the-art GNN training systems typically employ a hybrid CPU-GPU design where the frontend data preparation is undertaken by the CPU while the backend GNN training is handled by the GPU (Figure 3(a)). Because GPUs employ bandwidth-optimized but capacity-limited HBM, they are unable to locally store the memory capacity limited neighbor edge list array. Therefore, capacity-optimized CPU DIMMs are utilized for storing the memory-hungry graph data and the CPU goes through the neighbor sampling phase using the neighbor edge list array (step ① in Figure 1 and Figure 3(a)). Once the sampled subgraph is generated, the feature table is looked up to aggregate the corresponding feature vectors of each sampled node (step ②). The aggregated feature vectors are then copied over to the GPU over PCIe (step ③) for GNN training (step ④ and ⑤).

3 MOTIVATION AND CHARACTERIZATION

3.1 Motivation

A critical limitation with current graph learning frameworks (DGL [78] and PyG [21]) is that their *in-memory* processing model (i.e., the key data structures of large-scale GNN training must *all* be stored in DRAM) prevents developers from scaling up the graph structure and its feature vector size. One promising alternative is to employ NVMe SSDs for storing the memory-hungry data structures of GNN training (Figure 3(b)), utilizing main memory as a fast cache for high locality data. A potential challenge with SSDs, however, is that they operate as block devices where data is transferred in coarse 4 KB chunks at a much lower throughput than DRAM. In the remainder of this section, we conduct a detailed characterization on GraphSAGE based large-scale GNN training and identify

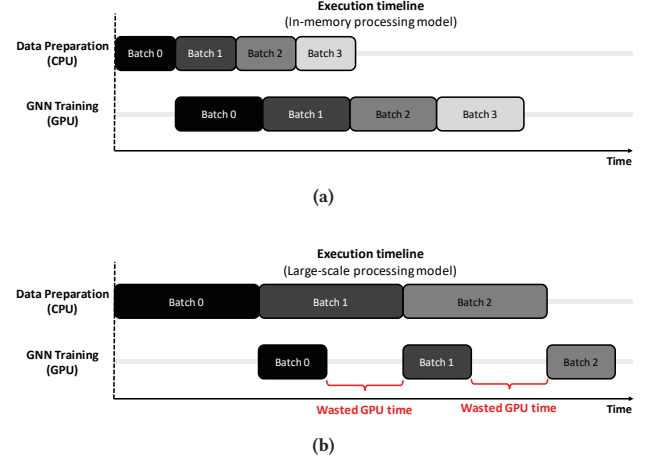


Figure 4: Execution timeline of hybrid CPU-GPU under (a) the baseline system assuming in-memory processing and (b) when the SSD is used to store the large-scale graph data. A single subgraph is generated by a CPU-side producer worker process, multiples of which are stored into the GPU work queue to be consumed by the GPU worker process sequentially for GNN training.

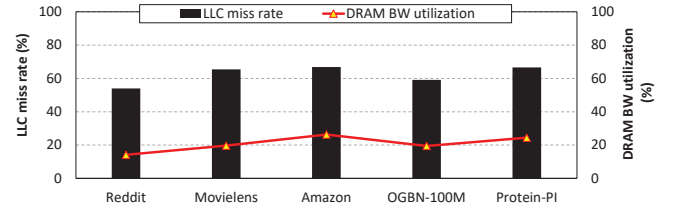


Figure 5: The LLC miss rate (left) and DRAM bandwidth utilization (right) during the neighbor sampling stage with baseline in-memory processing training using PyG. We utilize Linux perf (caching) and Intel RDT utility (bandwidth) for our evaluation.

key challenges of utilizing SSDs to address the memory capacity limitations of current in-memory processing ML frameworks.

3.2 Data Preparation in In-Memory Training

A typical GNN training pipeline employs the producer-consumer model as illustrated in Figure 4. The data preparation stage is implemented using multiple CPU-side producer workers operating in parallel, each of which conducts neighbor sampling to generate the mini-batch inputs, i.e., the subgraphs. These subgraphs are then stored into a work queue which the GPU-side consumer process utilizes to initiate GNN training.

The implication of storing the memory capacity limited graph datasets in an SSD is that the neighbor sampling stage of data preparation (step ① in Figure 1, Figure 3(b)) must now be conducted over the slow and low bandwidth I/O block interface. Understanding the algorithmic property of sampling and its memory access behavior is therefore crucial in gauging the feasibility of SSDs for large-scale GNN training.

To this end, in Figure 5, we start by first characterizing the on-chip caching and off-chip DRAM bandwidth utilization of the neighbor sampling algorithm under the baseline in-memory processing setting (i.e., all input graph dataset is assumed to be entirely

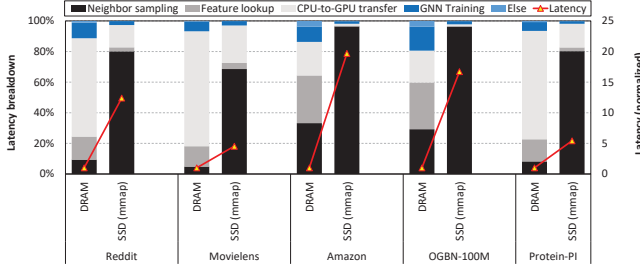


Figure 6: (Left-axis) Breakdown of GNN training time into key steps of data preparation (black-gray) and GNN training (blue). (Right-axis) End-to-end training time normalized to the baseline in-memory processing system (DRAM). Experiments are conducted using PyG.

stored in DRAM). Results show that neighbor sampling exhibits low caching efficiency with an average 62% last-level cache (LLC) miss rate. As discussed in Algorithm 1, neighbor sampling involves randomly selecting a fixed number of nearby nodes for each of the target nodes as it traverses through the k -hop neighborhoods. Because the target nodes that constitute a training mini-batch are typically scattered across the input graph, the execution of neighbor sampling is dominated by (random) memory lookup operations with little compute intensity, exhibiting a highly sparse and irregular dataflow. Interestingly, the off-chip memory bandwidth utilization is generally low despite such high memory intensity, consuming only an average 21% of 125 GB/sec maximum memory throughput. This is because each sampling operation only amounts to a fine-grained 8 byte read transaction, leading to severe under-utilization of DRAM read throughput. Such characterization result implies that the neighbor sampling algorithm is severely memory latency limited, rather than throughput limited, providing guidelines on optimizing our SSD based training system. We now explore the implication of conducting neighbor sampling over an SSD.

3.3 Data Preparation in SSD-centric Training

We establish our baseline SSD-centric training system to store the memory capacity limited graph dataset (esp. the neighbor edge list array) inside the SSD for the neighbor sampling operation (Figure 3(b)). These data structures are accessed using memory-mapped file I/O (mmap) which maps the contents of the graph dataset file within the user-space memory address. Because the most recently accessed pages are buffered inside main memory’s OS page cache, it is possible for our baseline SSD-centric system to significantly reduce the data fetch latency for high locality accesses, narrowing the performance gap between DRAM vs. SSD.

Figure 6 provides a breakdown of end-to-end GNN training time (left-axis) as well as normalized latency (right-axis) when comparing the baseline in-memory processing vs. mmap-based SSD-centric training system. As depicted, the baseline SSD-centric system incurs an average 9.8 $\times$ (maximum 19.6 $\times$) slowdown vs. in-memory processing. As discussed in Section 3.2, neighbor sampling exhibits fine-grained irregular parallelism with low locality. Therefore, the mmap-based SSD design point experiences significant misses in the OS page cache during neighbor sampling, so the merits of utilizing the page cache to reap locality benefits are outweighed by the high latency overheads of maintaining the OS managed page cache itself

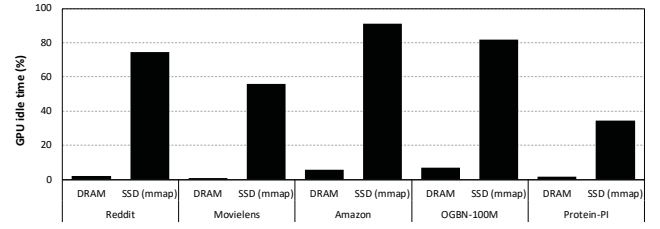


Figure 7: Percentage of training time where the GPU is idle due to the lack of input mini-batches to process.

(e.g., handling the page-faults incurred during the memory-mapped neighbor edge list array accesses, bringing in the faulted pages into the page cache, frequent user-kernel space context switches, etc). Consequently, the baseline SSD-centric system suffers from a throughput mismatch between the producer-consumer, leaving the GPU idle whenever the work queue runs out of subgraphs to use as inputs for GNN training. In Figure 7, we quantify the magnitude of such producer-consumer throughput mismatch by showing the fraction of training time where the GPU is left idle, waiting for the subgraphs to be generated by the CPU-side producer workers. The baseline in-memory processing model shows high GPU utilization because the data preparation stage is capable of generating input subgraphs at high throughput (Figure 4(a)). With the baseline mmap-based SSD-centric system, however, the producer workers fall short in sufficiently providing large enough number of subgraphs for the GPU to consume, exhibiting large periods of GPU idle time, causing a significant slowdown (Figure 4(b)).

Driven by our characterization, this paper explores an ISP based SSD-centric architecture for large-scale GNN training. As we detail in the next section, our proposition holistically addresses the dual challenges of memory capacity limited large-scale graph learning and the wide performance gap between DRAM vs. SSD.

4 SMARTSAGE ARCHITECTURE

4.1 Architecture Overview

Our proposed SmartSAGE architecture employs an ISP accelerator tailored for subgraph generation, co-designed with our latency-optimized software runtime system and host driver. This section takes a bottom-up approach in presenting SmartSAGE, describing our hardware level innovations first (Section 4.2) followed by a discussion of the software architecture that interfaces our proposed ISP architecture to the ML frameworks (Section 4.3).

Hardware. The SmartSAGE ISP is implemented as part of the firmware within the SSD (Figure 8). Such design decision allows SmartSAGE to maintain compatibility with existing hardware (SSD hardware) and software (the OS and the NVMe protocol) stack. By initiating neighbor sampling near SSD, however, SmartSAGE is able to increase *effective* throughput for subgraph generation by utilizing the internal SSD bandwidth. Because the sampled, subgraphs are densely packed within the SSD→DRAM returned logical blocks, SmartSAGE can significantly reduce unused data transferred over PCIe, achieving high neighbor sampling throughput.

Software. SmartSAGE is designed to reduce the command and control overheads in the OS and host driver stack by optimizing the software runtime for latency rather than locality. Instead of

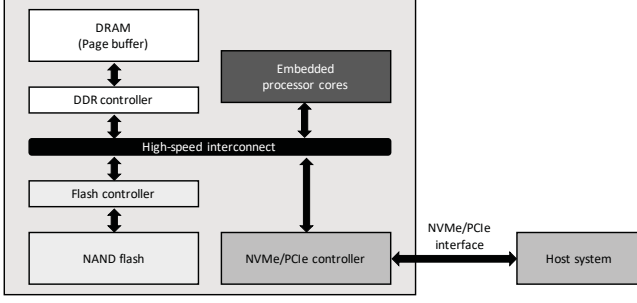


Figure 8: A firmware-based CSD architecture. The embedded processor cores execute routine SSD firmware tasks (e.g., FTL) to handle the host-side I/O block requests and generate flash page read/write requests from/to the flash devices, buffering them inside an on-device DRAM buffer (referred to as SSD’s DRAM page buffer) to send it back to the host CPU. SmartSAGE utilizes these embedded processor cores to run neighbor sampling directly off of SSD’s DRAM page buffer, seeking to reduce latency.

needlessly incurring several tens of microseconds of latency to maintain the opportunistic OS page cache, SmartSAGE allocates a user-space scratchpad buffer to *manually* orchestrate high locality data movements using Linux direct I/O. This allows SmartSAGE software runtime system to *bypass* the OS page cache thereby significantly reducing the latency overheads of traversing through the system software stack. Additionally, SmartSAGE host driver employs an ISP instruction that coalesces multiple I/O commands under a single NVMe command, further reducing latency. Such lightweight communication path is utilized by our SmartSAGE ISP architecture to streamline the subgraph generation process with high performance.

4.2 Hardware Acceleration using In-Storage Processing Architectures

Why choose firmware-based (and not FPGA-based) CSDs for ISP? SmartSAGE ISP unit accelerates the subgraph generation process by offloading the neighbor sampling operator as part of SSD’s firmware execution. There are currently two prominent approaches in designing a computational storage device (CSD) with ISP capabilities. One popular approach in designing CSDs is to utilize the embedded cores within the SSD device for in-storage processing at the SSD’s firmware level (e.g., OpenSSD [43], NGD system’s Newport [15, 71], Biscuit [25]). An alternative CSD design point is to utilize FPGA circuitry integrated near the SSD device and conduct ISP at the hardware level (e.g., Samsung’s SmartSSD [66], Eideticom’s NoLoad CSP [19]). In the remainder of this paper, we refer to each of these CSD types as *firmware-based* CSD and *FPGA-based* CSD, respectively.

We observe that offloading neighbor sampling to an FPGA-based CSD is not cost-effective because of the following two factors. First, as our characterization study revealed in Section 3, the neighbor sampling operator is mostly dominated by random data lookups with very low compute intensity. Therefore, synthesizing hardwired (FPGA) circuitry to merely conduct data gathers and scatters can underutilize the FPGA reconfigurable logic (although this problem can potentially be alleviated by spatially sharing the FPGA among different processes). But more crucially, neighbor sampling

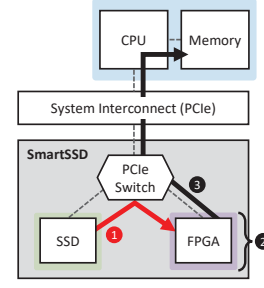


Figure 9: Key steps undertaken when conducting neighbor sampling over an FPGA-based CSD (e.g., SmartSSD [66]). Because the SSD and FPGA communicates over a PCIe switch integrated within the CSD device, neighbor sampling over an FPGA-based CSD incurs a two-step P2P transfer (i.e., SSD→FPGA and then FPGA→CPU), unlike a firmware-based CSD where a single SSD→CPU data transfer is invoked.

offloaded to an FPGA-based CSD incurs a two-step P2P data transfer which adds significant performance overhead: 1) SSD→FPGA to conduct the neighbor sampling using FPGA and generate the subgraph (step 1/2 in Figure 9), and 2) FPGA→CPU to transfer the subgraph to host CPU memory (step 3 in Figure 9). We observe that the latency overheads of such two-step neighbor sampling outweighs the benefits of ISP acceleration, providing little benefits over the baseline mmap-based SSD system.

To this end, SmartSAGE employs a firmware-based CSD substrate to implement our ISP architecture (Figure 8). Because neighbor sampling is mostly composed of irregular data lookups with low compute intensity, it suits well to the wimpy embedded cores (relative to the host-side x86 CPUs) equipped within firmware-based CSDs. Later in Section 6.4, we quantitatively evaluate both CSD design points for the completeness of our study.

Key observations and proposed approach. Figure 10 illustrates the key intuition behind SmartSAGE ISP neighbor sampling operator. Here, the neighbor edge list array stores all the neighborhood nodes’ IDs around a given graph node in a sequential manner, for *all* the nodes within the graph dataset. Under the baseline mmap-based SSD, the CPU initiates a memory-mapped block I/O read requests to this data structure as means to fetch all the target node’s neighborhood ID list into main memory. Because the target node IDs within a mini-batch are rarely aligned consecutively, there exists little spatial locality that can be reaped out by the CPU during neighbor sampling. Consequently, the baseline SSD-centric system ends up generating a large number of I/O block fetch requests, proportional to the number of target nodes subject for neighbor sampling (e.g., one edge list “chunk” is fetched per each target node in Figure 10(a)). This results in a significant overfetching of useless data from SSD→CPU, leading to severe underutilization of precious I/O bandwidth. SmartSAGE ISP architecture, on the other hand, performs fine-grained gather operations from the edge list array *directly* from the SSD’s DRAM page buffer, constructing a dense ID list of “sampled” neighborhood node IDs (Figure 10(b)). Because this dense, sampled node ID list (i.e., the subgraph itself) is subject for SSD→CPU transfer, SmartSAGE is able to significantly amplify the *effective* throughput of neighbor sampling.

Hardware/software interaction in ISP neighbor sampling. We now detail the key components of SmartSAGE ISP design as well

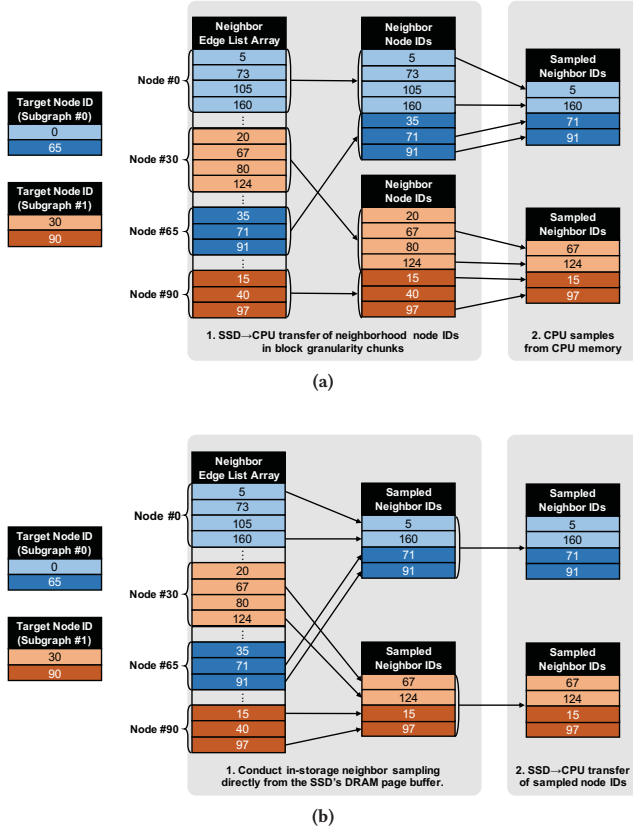


Figure 10: (a) Subgraph generation under the baseline SSD-centric system and (b) our SmartSAGE ISP based neighbor sampling operation, amplifying effective throughput in generating the subgraph. The example illustrates the process of extracting two subgraphs over two separate mini-batches (blue: mini-batch #0, orange: mini-batch #1), each generating a subgraph from two target nodes (target node ID of (blue: 0/65) and (orange: 30/90) and its neighborhood nodes (sampling rate: 2 neighbors).

as the associated hardware/software interactions involved during in-storage neighbor sampling (Figure 11). SmartSAGE adds two major components within the SSD firmware: 1) The ISP control unit handles the CPU→SSD offloaded subgraph generation request, generating the necessary set of commands to send to the subgraph generator. 2) The subgraph generator is in charge of extracting out the subgraph from the large-scale input graph, sending series of flash page read requests to the low-level flash devices. Once the requested flash pages are returned and subsequently cached into SSD's DRAM page buffer, the subgraph generator initiates in-storage neighbor sampling to gather the sampled nodes of the subgraph under construction. Below we summarize the major steps undertaken during SmartSAGE's subgraph generation.

- (1) Our custom designed SmartSAGE driver (detailed in Section 4.3) sends a subgraph generation request to the SSD firmware initially as an NVMe write command, which includes a pointer to the *neighbor sampling configuration data* (NS_{config}) stored inside CPU memory. NS_{config} contains key parameters of the sampling operation, e.g., number of target nodes as well as their logical block address, neighborhood node IDs to sample, and other metadata (step ① in Figure 11).

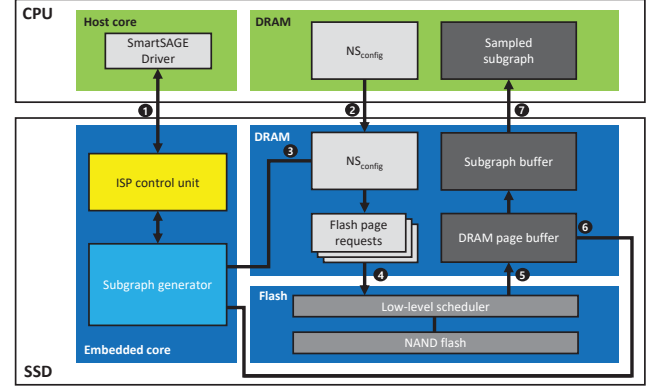


Figure 11: Major components of SmartSAGE ISP architecture and the key steps undertaken during the lifetime of in-storage neighbor sampling.

- (2) When the SSD firmware receives the subgraph generation request, it triggers a DMA access to CPU memory to copy the NS_{config} data from CPU→SSD using SSD's NVMe host controller (step ②). Once the SSD receives the NS_{config} data, it goes through several stages to prepare for subgraph generation, one major step being the address translation process (from the logical address to flash's physical page address) to determine where within the flash devices should the subgraph generator send flash page read requests to (step ③). A given target node's neighbor nodes' ID list can potentially require multiple flash page read requests depending on the number of edges connected to the target node.
- (3) The flash page read requests are sent to the pending flash page request queue to prepare for in-storage neighbor sampling, which the low-level flash controller utilizes to kick off flash page read operations (step ④).
- (4) Once a flash page read request is serviced, the corresponding neighbor edge list will be cached inside SSD's DRAM page buffer (step ⑤). SmartSAGE utilizes the SSD's embedded cores to conduct fine-grained neighbor sampling over the neighbor edge list, stored inside the SSD's DRAM page buffer. The sampled nodes are collected into ISP's pending subgraph buffer (step ⑥).
- (5) Once all the target nodes' neighbor IDs are sampled, the sampled subgraph is ready to be transferred back to the CPU. At the high level of the SSD firmware polling loop, our scheduler checks for completed subgraph generation requests. If the sampled subgraph is ready and the NVMe host controller is available, we initiate a SSD→CPU DMA write operation to copy back the subgraph information into CPU DRAM (step ⑦).

4.3 Latency-optimized Runtime and Host Driver Stack

Although our ISP design helps significantly reduce the latency of neighbor sampling, there are still significant performance improvement opportunities to be reaped out. As discussed in Section 3, the baseline mmap-based SSD experiences significant slowdown as the locality-optimized OS page cache is rarely useful in reducing I/O

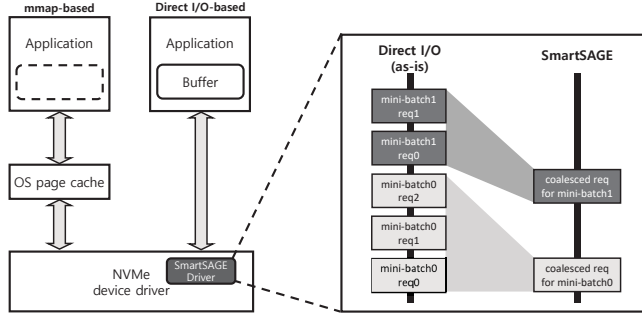


Figure 12: SmartSAGE’s latency-optimized direct I/O based software runtime (left) and host driver stack employing I/O command coalescing (right).

access time. Additionally, recall from Figure 10 that the baseline SSD invokes a large number of I/O block fetch requests per each mini-batch training as the data accesses to the neighbor edge list exhibit low spatial locality (i.e., only a single edge list “chunk” can be requested over a single I/O block fetch request), which adds another layer of latency penalty. This is because servicing any given I/O block fetch request involves context switching through the user↔kernel space when traversing through the host driver stack.

We design our software runtime and the host driver stack to be optimized for latency first and locality second, as detailed below.

Direct I/O. Linux comes with a direct I/O feature where the file read/write operations go directly from the user-space application to the storage device, *bypassing* the OS page cache completely (Figure 12). Given the locality limited nature of neighbor sampling, our runtime system utilizes such feature (`O_DIRECT` flag) to allocate a user-space buffer to *manually* orchestrate high locality data movements between SSD↔CPU, without relying on the opportunistic OS page cache. As we quantitatively analyze in Section 6.2, our direct I/O based software runtime alone, even without the ISP feature, helps reduce neighbor sampling latency by 2.9× compared to the baseline mmap-based SSD system.

I/O command coalescing. As detailed in Section 4.2, the in-storage neighbor sampling operation is invoked by sending a subgraph generation request in the form of an NVMe write command to the SSD. Unlike the baseline SSD-centric system which spawns off multiple high latency I/O fetch requests for a single subgraph generation, SmartSAGE’s host driver encapsulates the *entire* gather/scatter operation of neighbor sampling under a *single* NVMe transaction. More concretely, all the key parameters of subgraph generation (e.g., logical block address of all target nodes, neighborhood node ID lists to sample, etc) are stored as part of the NS_{config} data (Figure 11), which the SSD firmware retrieves through a one-time CPU→SSD DMA transaction. This allows SmartSAGE to significantly reduce the number of I/O commands invoked to generate a single subgraph, reducing command and control overheads of the host driver stack and further reducing latency.

NVMe compatibility and system integration. Our custom I/O interface is implemented using the `ioctl()` system call, which maintains complete compatibility with current NVMe protocols. The subgraph generation request is indicated using a single unused command bit, which the SSD firmware utilizes to invoke in-storage neighbor sampling. Aside from this special-purpose bit, the hardware/software interface utilizes the exact same command structure

Table 1: Graph dataset information.

Dataset	In-memory			Large-scale			Features
	Nodes	Edges	Size (GB)	Nodes	Edges	Size (GB)	
Reddit	233.0K	114.6M	0.8	37.3M	53.9B	402	602
Movielens	5.5M	6.0B	45	22.2M	59.2B	442	1K
Amazon	42.5M	1.3B	9.7	265.9M	9.5B	75	32
OGBN-100M	89.6M	3.2B	26	179.1M	5.0B	41	32
Protein-PI	907.0K	317.5M	2.4	9.1M	8.8B	66	512

of conventional I/O read/write commands and the CPU→SSD copy of NS_{config} data as well as SSD→CPU copy of the final, neighbor sampled subgraph is orchestrated using existing DMA engines.

Because SmartSAGE’s runtime and host driver stack maintains compatibility with current NVMe protocol and the ISP operator is implemented as part of the SSD firmware, SmartSAGE is fully compatible with existing CSD architectures.

5 METHODOLOGY

Hardware/software platform. We use PyTorch Geometric to implement a GraphSAGE based GNN training pipeline. When evaluating end-to-end training performance, we employ a CPU-GPU system containing an Intel Xeon Gold 6242 CPU with 192 GB of DRAM and NVIDIA’s Tesla T4 GPU.

CSD platform. We implement our neighbor sampling ISP operator using the open-source Cosmos+ OpenSSD [43]. OpenSSD contains a fully functional NVMe flash SSD with 2 TB of storage and a customizable SSD firmware executed by a dual core ARMv7 Cortex-A9 processor. The host interface controller of OpenSSD communicates with the CPU over an 8-lane PCIe (gen2) channel. We design SmartSAGE’s ISP operator within OpenSSD’s firmware and measure wall clock time for evaluating performance.

Comparison to alternative training systems. While we demonstrate SmartSAGE’s merits using OpenSSD, for the completeness of our study, we also evaluate SmartSAGE’s ISP neighbor sampling operator on top of an FPGA-based CSD using Samsung-Xilinx’s SmartSSD [8] (Section 6.4). We also compare SmartSAGE against a training system utilizing Intel’s Optane DC Persistent Memory Module (PMEM) that is installed on the memory bus (NVDIMM), providing 768 GB of capacity to store the graph datasets (Section 6.3).

Dataset. Existing graph datasets utilized for GNN studies are generally at small-scale that comfortably fit within main memory, defeating the main purpose of our study. Prior work on Kronecker Graph Theory [7] suggests graph fractal expansion methods to expand the scale of the input graph dataset while properly maintaining the graph’s distinctive characteristics (e.g., power-law degree distribution, community structure, etc). We therefore employ Kronecker fractal expansion method [7] to synthetically generate large-scale graph datasets, the key features of which are summarized in Table 1 (the “Large-scale” column). Prior work [53] observed that real-world graphs that become larger with more nodes and edges generally exhibit higher average degrees (i.e., the rate at which the number of edges increase is faster than the increase in number of nodes), a property known as the *densification power law*. The synthetically generated large-scale datasets are designed to properly reflect such behavior, represented by the higher average degree of large-scale datasets vs. in-memory datasets (Figure 13). The graph datasets

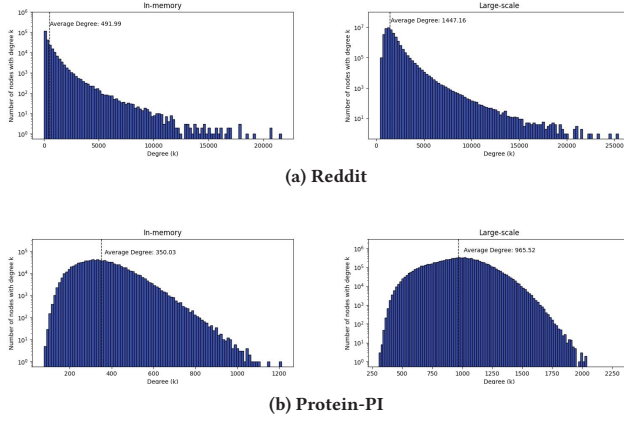


Figure 13: Degree distribution of the in-memory (left) vs. large-scale (right) graph datasets for a subset of our datasets. As depicted, despite Kronecker fractal expansion enabling the number of nodes as well as edges to increase dramatically (represented by the increase in the number of nodes with degree-K in the y-axis), the overall power-law distribution of each dataset, before/after fractal expansion is applied remains similar.

are all compressed in CSR (compressed sparse row) format and we employ the default configuration of GraphSAGE when training our GNN algorithm with a mini-batch size of 1024. In Section 6.6, we discuss the sensitivity of SmartSAGE when deviating from these default configurations.

6 EVALUATION

This section first focuses on exploring the following three design points: 1) the baseline mmap-based SSD training system (SSD(mmap)), and two design points employing our hardware/software optimizations, namely 2) direct I/O based SSD training system *without* ISP (SmartSAGE(SW)) and 3) our proposed architecture with all the proposed optimizations in place (SmartSAGE(HW/SW)). Later in Section 6.3, we compare SmartSAGE against an upper bound, oracular in-memory processing design point which assumes the CPU contains *infinite* DRAM capacity so that all the graph datasets can be stored locally in DRAM. SmartSAGE is also compared against Intel’s Optane DC PMEM in Section 6.3 and an ISP architecture based on an FPGA-based CSD in Section 6.4.

6.1 “Single” Worker’s Neighbor Sampling Performance

As discussed in Figure 4, a GNN training pipeline employs a producer-consumer model where multiple CPU-side workers independently conduct neighbor sampling for subgraph generation. To precisely understand the efficacy of our proposal on improving a CPU-side worker’s neighbor sampling operation, we first instantiate just a single worker for subgraph generation and evaluate its performance (Figure 14). Compared to the baseline mmap-based SSD system, our software-only SmartSAGE with direct I/O runtime (SmartSAGE(SW)) alone provides an average 1.5 $\times$ speedup in neighbor sampling. Such result demonstrates the advantage of using direct I/O to bypass the OS page cache, which helps optimize the data preparation stage for latency, rather than locality.

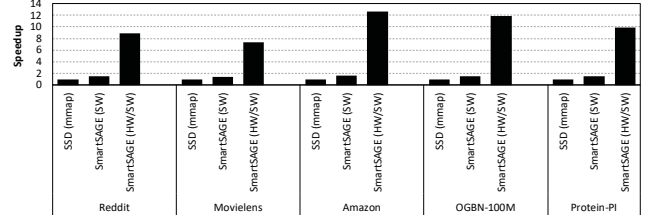


Figure 14: SmartSAGE’s speedup for neighbor sampling vs. baseline mmap-based SSD system (single worker).

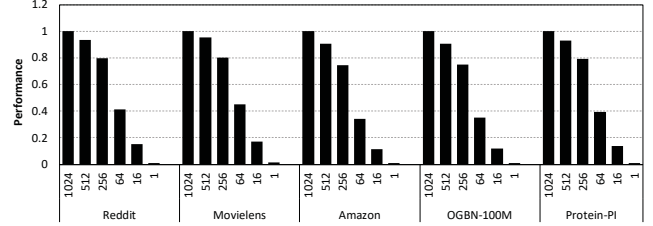


Figure 15: Effect of reducing the I/O command coalescing granularity on SmartSAGE(HW/SW) speedup.

The neighbor sampling performance is enhanced even further with SmartSAGE’s ISP architecture (SmartSAGE(HW/SW)), providing an additional average speedup of 6.6 $\times$ (maximum 7.8 $\times$) over the software-only SmartSAGE(SW). Overall, SmartSAGE(HW/SW) achieves an average 10.1 $\times$ (maximum 12.6 $\times$) speedup vs. the mmap-based SSD system, successfully resolving the bottlenecks of the baseline architecture.

It is worth mentioning that the significant speedup SmartSAGE achieves is not just a result of the hardware-level acceleration of neighbor sampling or direct I/O based software runtime; also equally important is our co-designed host driver stack that minimizes the I/O command and control overheads. In Figure 15, we show the effect of reducing the I/O command coalescing granularity on SmartSAGE(HW/SW)’s speedup. The default configuration of SmartSAGE(HW/SW) coalesces *all* the target nodes’ neighbor sampling within a given mini-batch (i.e., 1024 target nodes, the leftmost design point) under a *single* NVMe command, which is encapsulated inside a single NS_{config} (neighbor sampling configuration data, see Figure 11). As the coalescing granularity becomes smaller (from left to right in the x-axis), the latency overheads of sending the I/O commands to the SSD start outweighing the benefits provided with ISP, experiencing a significant performance hit.

Overall, the evaluation in this section highlights the effectiveness of SmartSAGE’s software/hardware co-design; 1) direct I/O, 2) I/O command coalescing, and 3) ISP acceleration.

6.2 “Multiple” Workers’ Neighbor Sampling Performance

We now explore the effectiveness of SmartSAGE when multiple workers are concurrently accessing the storage system for subgraph generation (Figure 16). SmartSAGE(HW/SW) provides an average 4.4 $\times$ (maximum 5.5 $\times$) speedup in neighbor sampling compared to the baseline SSD(mmap). Compared to the single worker execution scenario (Figure 14), however, notice that the additional speedup provided with SmartSAGE(HW/SW) vs. SmartSAGE(SW)

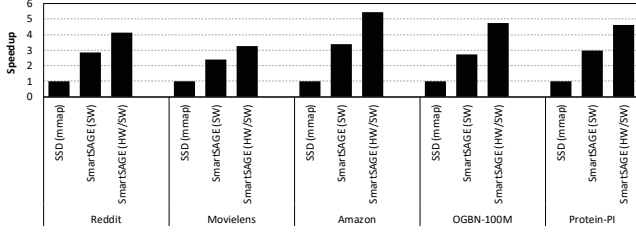


Figure 16: SmartSAGE’s neighbor sampling speedup vs. baseline SSD(mmap) (multiple workers). Results assume 12 concurrent workers as performance is at its highest with 12 workers, for both baseline and SmartSAGE.

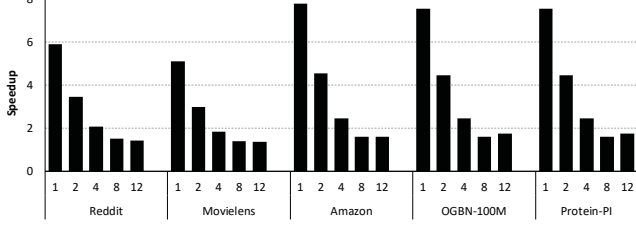


Figure 17: SmartSAGE(HW/SW)’s speedup vs. SmartSAGE(SW) when scaling up the number of CPU-side workers, from 1 to 12.

is reduced. Our analysis showed that, when multiple workers simultaneously access the SSD for in-storage neighbor sampling, OpenSSD’s relatively wimpy embedded cores get overwhelmed in delivering sufficient levels of ISP compute power for high throughput neighbor sampling. More concretely, since our OpenSSD based neighbor sampling operator *time-shares* the embedded cores with the flash management firmware, the interference our neighbor sampling causes to the SSD firmware degrades the level of speedup achieved with in-storage neighbor sampling. Such phenomenon is shown in Figure 17, where SmartSAGE(HW/SW)’s speedup vs. SmartSAGE(SW) gets gradually reduced as we add more CPU-side workers. Nonetheless, we emphasize that our evaluation results are highly conservative as state-of-the-art CSD architectures provide far more performance than our OpenSSD platform. For instance, recently announced CSDs like NGD system’s Newport [15, 71] provides much faster compute and communication performance for ISP usage. Unlike our OpenSSD platform which utilizes a dual core ARMv7 Cortex-A9 for firmware execution, Newport contains four ARMv7 M7 cores for firmware execution with an additional quad core ARMv8 Cortex-A53 solely dedicated for ISP purposes. Due to limited public availability of Newport CSDs, we employed the OpenSSD system as a proof of concept to our proposal, demonstrating substantial improvements in throughput vs. baseline SSD.

In general, we conclude that the SmartSAGE’s key insights are directly applicable to any firmware-based CSD because SmartSAGE maintains full compatibility with current NVMe protocol and its ISP is programmed at the software level as part of the SSD firmware.

6.3 End-to-End GNN Training Time

Figure 18 summarizes the effectiveness of SmartSAGE on reducing end-to-end GNN training time. To thoroughly cover the evaluation space, we explore two in-memory processing based systems: 1) Intel’s Optane DC Persistent Memory Module (PMEM) which can store the entire graph dataset within its NVDIMMs and 2) an

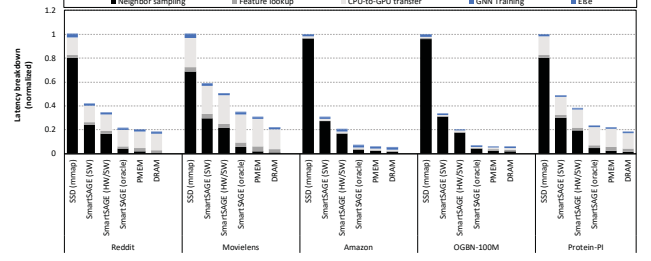


Figure 18: Latency breakdown of end-to-end GNN training time.

oracular DRAM-only design which assumes the main memory is infinitely sized to enable large-scale GNN training at DRAM speed. We also establish a hypothetical SmartSAGE design point which assumes that the CSD architecture contains dedicated, ISP-purposed embedded cores (like Newport CSD [15, 71]) such that the opportunities inherent with our in-storage neighbor sampling can be fully unlocked (SmartSAGE(oracle)). SmartSAGE(oracle)’s performance is estimated by assuming the neighbor sampling speedup achieved under single worker execution (Figure 14) can equally be achieved under multiple workers.

Compared to the baseline mmap-based SSD system, SmartSAGE (HW/SW) provides an average $3.5\times$ (maximum $5.0\times$) improvement in training throughput. While such improvements are impressive, SmartSAGE(HW/SW) still incurs an average 60% performance loss vs. the DRAM-only design point. Nonetheless, such DRAM-only architecture is an unbuildable, upper bound design point and existing in-memory processing based systems are not able to train the large-scale GNNs that our SSD-based system will enable. Intel PMEM does much better than the baseline SSD(mmap), “only” incurring an average $1.2\times$ slowdown vs. the oracular DRAM-only design. However, the performance advantage of PMEM comes at the cost of lower storage density and lower GB/\$ vs. SSD. Compared to these high cost in-memory processing based systems, SmartSAGE(oracle) performs very competitively, achieving an average 70% and 90% of the performance of the DRAM-only and PMEM based systems, respectively. These results demonstrate that, with newer/future CSD architectures provisioned with more ISP compute power and faster flash devices, an NVMe SSD based system can become a viable option for large-scale GNN training while not compromising on performance.

6.4 Comparison to FPGA-based CSD Designs

In Section 4.2, we argued that a firmware-based CSD is much more appropriate for in-storage neighbor sampling compared to an FPGA-based CSD. Figure 19 quantitatively demonstrates our rationale behind such design decision. We utilize Samsung-Xilinx’s SmartSSD [66] to implement a neighbor sampling operator within SmartSSD’s FPGA logic (denoted “FPGA-CSD”). Such an FPGA-CSD based design conducts in-storage neighbor sampling by: 1) copying the necessary chunks of neighbor edge list array from SSD→FPGA using P2P transfer (gray), 2) utilizing FPGA’s hardwired neighbor node gather unit to conduct sampling over FPGA’s local DRAM (yellow), and 3) transferring the sampled subgraph from FPGA→CPU (blue). As depicted, the performance of FPGA-CSD is bottlenecked on the latency to move the neighbor edge list array chunks from

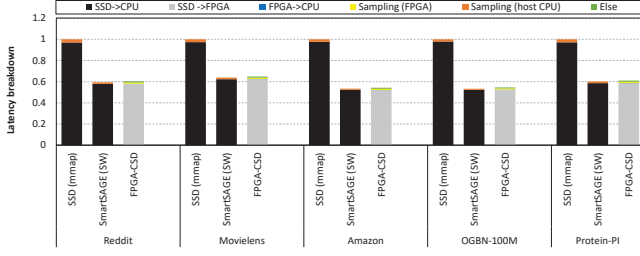


Figure 19: FPGA-based CSD vs. SSD(mmap) and SmartSAGE(SW).

SSD→FPGA, failing to achieve any performance advantage even over our software-only SmartSAGE(SW).

6.5 Power and Energy Consumption

The CPU-GPU based GNN training system consumes several thousands of watts of system-wide power (i.e., the CPU, GPU, DRAM, SSD). Due to the following factors, the added power overheads of SmartSAGE is expected to be negligible, allowing the significant reduction in training time to proportionally improve system-level energy-efficiency. First, SmartSAGE(HW/SW) is a purely firmware-based CSD that utilizes existing embedded cores within the SSD. Consequently, the added power overheads of SmartSAGE(HW/SW), if any, is amortized by the significant reduction in end-to-end training time. For the more future-looking SmartSAGE(oracle) design that integrates additional, *dedicated* ISP-purposed embedded cores (assuming quad core ARMv8 Cortex-A53 like NGD Newport), the CSD will add around 2 – 6 watts of TDP of additional power consumption, which is a reasonable overhead (vs. CPU/GPU/DRAM/SSD’s system-level power consumption) given the significant reduction in training time.

6.6 Sensitivity

This subsection evaluates SmartSAGE’s robustness to various configuration parameters of GNN training.

Alternative sampling algorithm. To demonstrate the robustness of our proposal, we implement another state-of-the-art graph sampling algorithm, GraphSAINT [88]. Unlike GraphSAGE, GraphSAINT employs a regular random walk based method to sample a neighbor per node across multiple target nodes. As depicted in Figure 20, SmartSAGE achieves an average of 8.2× end-to-end speedup with GraphSAINT, demonstrating our proposal’s robustness and flexibility in accommodating various graph sampling algorithms.

Sampling rate. Figure 21 shows SmartSAGE’s neighbor sampling speedup when we sweep the sampling rate by 0.5× and 2× of the default setting (default: 25 and 10 neighbor nodes sampling per each target node in the first and second GNN layer, respectively). As depicted, SmartSAGE(HW/SW)’s speedup is gradually decreased (increased) as sampling rate gets larger (smaller). Under SmartSAGE(HW/SW), as we increase the sampling rate, the subgraph being transferred over SSD→CPU gets larger and eventually approaches the data transfer size of SmartSAGE(SW). Such phenomenon renders SmartSAGE(HW/SW)’s speedup to become smaller than when evaluated under the default sampling rate.

Training batch size. We also study SmartSAGE’s sensitivity to larger/smaller mini-batch sizes. Results showed that the chosen mini-batch size have little effect on SmartSAGE’s achieved speedup.

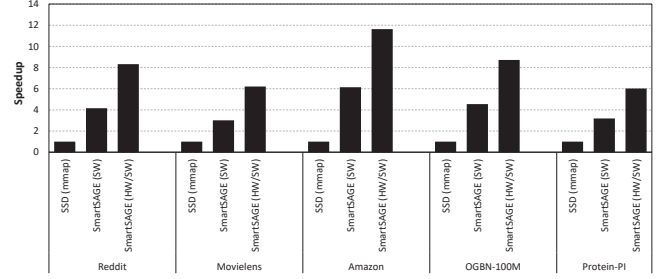


Figure 20: Sensitivity of SmartSAGE’s speedup to different sampling algorithm, i.e., GraphSAINT.

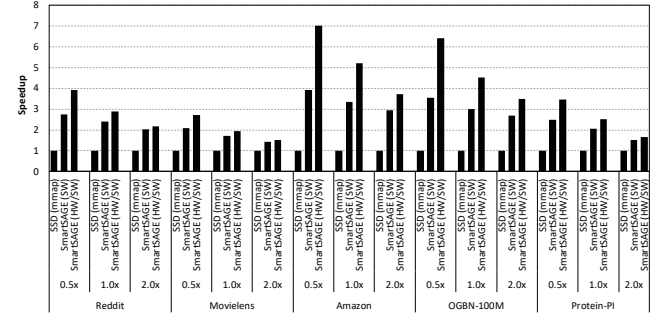


Figure 21: Sensitivity of SmartSAGE’s end-to-end speedup to sampling rate.

In general, we confirmed SmartSAGE’s robustness under different design points but omit the results due to space constraints.

7 RELATED WORK

The focus of our work is on accelerating large-scale GNN training with an ISP architecture. There is a large body of prior literature exploring in-storage/near-data processing [1, 5, 10, 12, 13, 16, 25, 27, 31–33, 35, 38, 38, 39, 42, 49, 52, 54, 58, 65, 67, 72–74, 76, 81, 81–83] or in-memory processing [2–4, 9, 18, 20, 23, 29, 34, 36, 37, 44, 45, 50, 60, 68, 79] architectures for data-intensive workloads as well as ASIC/FPGA/GPU based acceleration for graph neural networks [24, 40, 51, 54–56, 59, 69, 80, 84, 87]. There is also prior work exploring heterogeneous memory systems for training large-scale ML algorithms [11, 28, 30, 46–48, 61–64, 77]. Due to space limitations, we summarize a subset of these related work below.

ISP designs for data-intensive workloads. BlueDBM [31] investigated an ISP architecture targeting big data analytics, where a dedicated FPGA fabric is utilized for accelerating nearest neighbor search, string search, and graph traversal. Biscuit [25] similarly explores ISP acceleration for big data workloads but the ISP substrate utilized is more close to a firmware-based CSD rather than BlueDBM’s FPGA-based CSD design. Active Flash [72], Catalina [73], ActiveSort [52], Morpheus [74], GraFBoost [32], and work by Do et al. [16] are also ISP architectures that target data analytics (e.g., pattern matching, k -mean clustering, etc), database management, MapReduce frameworks, object (de)serialization, graph analytics, and Microsoft SQL, respectively. Aside from these prior art focusing on domain-specific acceleration of a particular application area, there is also a set of previous work that seeks to enhance the programmability of ISP designs. Willow [67] explores architectural

support to enhance the programmability and flexibility of using programmable SSDs for ISP. Summarizer [42] proposes a set of flexible programming APIs for conducting ISP-based data filtering and data summarization operations. GraphSSD [58] proposes a graph semantic aware ISP architecture that provides a simple programming interface for enhanced programmability. The adoption of CSDs for domain-specific acceleration is a trait SmartSAGE resembles to these prior literature, especially with Biscuit, Summarizer, and GraphSSD. Unlike SmartSAGE, however, Biscuit employs a dedicated pattern matching IP for ISP, which cannot be utilized for GNN's graph sampling. Similarly, GraphSSD is specifically optimized for efficient graph data layout mapping within the physical pages, focusing on in-storage graph update operators for bandwidth amplification, which cannot be readily employed for graph sampling. Summarizer bears similarity to SmartSAGE in that it supports in-storage reduction operations for generic data-intensive workloads like TPC-H. Because Summarizer does not target GNN training, however, it lacks several of SmartSAGE's software-level optimizations like direct I/O or mini-batch level I/O command coalescing (Figure 12). More crucially, our work targets a new, emerging application domain (i.e., machine learning based graph neural network training) and uncovers a new system-level bottleneck driven by different intuitions, rendering our key contributions unique.

Accelerating GNN inference. There is also a rich set of previous work exploring hardware/software acceleration techniques for machine learning inference [24, 38, 40, 55, 56, 69, 80, 81, 84]. While not specifically targeting GNNs, RecSSD [81] and work by Kim et al. [38] are recent ISP designs utilizing firmware-based CSDs to overcome the memory capacity bottlenecks of the inference stage of DNN-based recommendation models. HyGCN [84] is one of the first ASIC based GCN accelerators providing substantial energy-efficiency improvement for the backend GNN layers of graph learning. GLIST [54] is an FPGA-based CSD for GNN inference and GNNAdvisor [80] proposes an efficient software runtime for GPU-based GNN execution. Unlike these prior work focusing on accelerating the inference of GNNs, SmartSAGE focuses on uncovering the system-level bottlenecks of training, more specifically the frontend data preparation stage. Overall, the key contribution of SmartSAGE is orthogonal to these prior studies.

Accelerating GNN training. GraphACT [87] proposes a CPU-FPGA based acceleration platform for GNN training. Marius [59] is a software framework that aims to provide high-performance large-scale graph learning within a single machine. Unlike SmartSAGE however, Marius targets a different graph learning algorithm so the model parameters trained with Marius are the graph embedding vectors, not the GNN layers (as is the case with SmartSAGE). More importantly, Marius proposes a software-level pipelining solution for fast training, unlike the ISP based SmartSAGE. Similar to SmartSAGE, prior work on AliGraph/PaGraph/DistDGL [57, 89, 90] employ off-the-shelf graph partitioning algorithms for large-scale GNN training, partitioning the graph sampling and learning process across multiple workers for parallel GNN training. As pointed out by Su et al. [70], distributed GNN training can suffer from load imbalance issues (e.g., number of k -hop sampled nodes can differ across graph partitions), intermittent model parameter synchronization overheads, and etc. SmartSAGE explores a different research space for GNN training systems, enabling large-scale GNN training

within a single machine using NVMe SSD while still achieving DRAM-level performance. Overall, the contribution of SmartSAGE is orthogonal to these studies.

8 CONCLUSION

In this work, we investigate the viability of utilizing NVMe SSDs to overcome the memory capacity limitations of current, in-memory processing GNN training systems. We propose an in-storage processing based GNN training system called SmartSAGE which synergistically combines the ISP capabilities of emerging CSDs with a latency-optimized software runtime and host driver stack. By intelligently offloading the data intensive frontend data preparation stage of GNN training, SmartSAGE significantly resolves the bottlenecks of the baseline SSD-centric training system, achieving significant performance improvements.

ACKNOWLEDGMENTS

This research is partly supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government(MSIT) (NRF-2021R1A2C2091753), the Super Computer Development Leading Program of the NRF funded by the Korea government MSIT under grant NRF-2020M3H6A1085498, and by Samsung Electronics Co., Ltd (IO201210-07974-01). We also appreciate the support from Samsung Advanced Institute of Technology (SAIT) and the EDA tools supported by the IC Design Education Center (IDEC), Korea. Minsoo Ryu is the corresponding author.

REFERENCES

- [1] Anurag Acharya, Mustafa Uysal, and Joel Saltz. 1998. Active Disks: Programming Model, Algorithms and Evaluation. *ACM SIGOPS Operating Systems Review* (1998).
- [2] Shaheen Aga, Supreet Jeloka, Arun Subramaniyan, Satish Narayanasamy, David Blaauw, and Reetuparna Das. 2017. Compute Caches. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*.
- [3] Mohammad Alian, Seung Won Min, Hadi Asgharimoghaddam, Ashutosh Dhar, Dong Kai Wang, Thomas Roewer, Adam McPadden, Oliver O'Halloran, Deming Chen, Jinjun Xiong, Daehoon Kim, Wen-mei Hwu, and Nam Sung Kim. 2018. Application-Transparent Near-Memory Processing Architecture with Memory Channel Network. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [4] Hadi Asghari-Moghaddam, Young Hoon Son, Jung Ho Ahn, and Nam Sung Kim. 2016. Chameleon: Versatile and Practical Near-DRAM Acceleration Architecture for Large Memory Systems. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [5] Duck-Ho Bae, Jin-Hyung Kim, Sang-Wook Kim, Hyunok Oh, and Chanik Park. 2013. Intelligent SSD: A Turbo for Big Data Mining. In *Proceedings of the ACM International Conference on Information & Knowledge Management*.
- [6] Trapit Bansal, David Belanger, and Andrew McCallum. 2016. Ask the GRU: Multi-task Learning for Deep Text Recommendations. In *Proceedings of the ACM Conference on Recommender Systems (RECSYS)*.
- [7] Francois Belletti, Karthik Lakshmanan, Walid Krichene, Yi-Fan Chen, and John Anderson. 2019. Scalable Realistic Recommendation Datasets through Fractal Expansions. *arXiv preprint arXiv:1901.08910* (2019).
- [8] Woosung Cheong, Chanhoo Yoon, Seonghoon Woo, Kyuwook Han, Daehyun Kim, Chulseung Lee, Youra Choi, Shine Kim, Dongku Kang, Geunyeong Yu, Jaehong Kim, Jaechun Park, Ki-Whan Song, Ki-Tae Park, Sangyeun Cho, Hwaseok Oh, Daniel DG Lee, Jin-Hyeok Choi, and Jaeheon Jeong. 2018. A Flash Memory Controller for 15us Ultra-Low-Latency SSD Using High-Speed 3D NAND Flash with 3us Read Time. In *Proceedings of the International Solid State Circuits Conference (ISSCC)*.
- [9] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [10] Benjamin Y Cho, Won Seob Jeong, Doohwan Oh, and Won Woo Ro. 2013. XSD: Accelerating MapReduce by Harnessing the GPU Inside an SSD. In *Proceedings of the 1st Workshop on Near-Data Processing*.

- [11] M. Cho, T. Le, U. Finkler, H. Imai, Y. Negishi, T. Sekiyama, S. Vinod, V. Zolotov, K. Kawachiya, D. Kung, and H. Hunter. 2018. Large Model Support for Deep Learning in Caffe and Chainer. In *SysML*.
- [12] Sangyeun Cho, Chanik Park, Hyunok Oh, Sungchan Kim, Youngmin Yi, and Gregory R Ganger. 2013. Active Disk Meets Flash: A Case For Intelligent SSDs. In *Proceedings of the ACM International Conference on Supercomputing (ICS)*.
- [13] I Stephen Choi and Yang-Suk Kee. 2015. Energy Efficient Scale-In Clusters with In-Storage Processing for Big-Data Analytics. In *Proceedings of the International Symposium on Memory Systems (MEMSYS)*.
- [14] Hanjun Dai, Zornitsa Kozareva, Bo Dai, Alex Smola, and Le Song. 2018. Learning Steady-States of Iterative Algorithms over Graphs. In *ICML*. 1114–1122.
- [15] Jaeyoung Do, Victor C. Ferreira, Hossein Bobarshad, Mahdi Torabzadehkashi, Siavash Rezaei, Ali Heydarigorji, Diego Souza, Bruno F. Goldstein, Leandro Santiago, Min Soo Kim, Priscila M. V. Lima, Felipe M. G. Franca, and Vladimir Alves. 2020. Cost-Effective, Energy-Efficient, and Scalable Storage Computing for Large-Scale AI Applications. *ACM Transactions on Storage* (2020).
- [16] Jaeyoung Do, Yang-Suk Kee, Jignesh M Patel, Chanik Park, Kwanghyun Park, and David J DeWitt. 2013. Query Processing on Smart SSDs: Opportunities and Challenges. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (MOD)*.
- [17] David Duvenaud, Dougal Maclaurin, Jorge Aguilera-Iparraguirre, Rafael Gomez-Bombarelli, Timothy Hirzel, Alan Aspuru-Guzik, and Ryan P. Adams. 2015. Convolutional Networks on Graphs for Learning Molecular Fingerprints. In *Proceedings of the International Conference on Neural Information Processing Systems (NIPS)*.
- [18] Charles Eckert, Xiaowei Wang, Jingcheng Wang, Arun Subramanian, Ravi Iyer, Dennis Sylvestre, David Blaauw, and Reetuparna Das. 2018. Neural Cache: Bit-Serial In-Cache Acceleration of Deep Neural Networks. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [19] Eideticom 2021. NoLoad CSP. <https://www.eideticom.com/products.html>
- [20] Amin Farmahini-Farahani, Jung Ho Ahn, Katherine Morrow, and Nam Sung Kim. 2015. NDA: Near-DRAM Acceleration Architecture Leveraging Commodity DRAM Devices and Standard Memory Modules. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*.
- [21] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [22] Alex Fout, Jonathon Byrd, Basir Shariat, and Asa Ben-Hur. 2017. Protein Interface Prediction Using Graph Convolutional Networks. In *Proceedings of the International Conference on Neural Information Processing Systems (NIPS)*.
- [23] Mingyu Gao, Jing Pu, Xuan Yang, Mark Horowitz, and Christos Kozyrakis. 2017. TETRIS: Scalable and Efficient Neural Network Acceleration with 3D Memory. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [24] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, T. Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, S. Reinhardt, and M. Herboldt. 2020. AWB-GCN: A Graph Convolutional Network Accelerator with Runtime Workload Rebalancing. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [25] Boncheol Gu, Andre S. Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, Jaeheon Jeong, and Duckhyun Chang. 2016. Biscuit: A Framework for Near-Data Processing of Big Data Workloads. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [26] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Proceedings of the International Conference on Neural Information Processing Systems (NIPS)*.
- [27] Yu-Ching Hu, Murtuza Taher Lokhandwala, Te I., and Hung-Wei Tseng. 2019. Dynamic Multi-Resolution Data Storage. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [28] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [29] Ranggi Hwang, Taehun Kim, Youngeun Kwon, and Minsoo Rhu. 2020. Centaur: A Chiplet-based, Hybrid Sparse-Dense Accelerator for Personalized Recommendations. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [30] Hai Jin, Bo Liu, Wenbin Jiang, Yang Ma, Xuanhua Shi, Bingsheng He, and Shaofeng Zhao. 2018. Layer-Centric Memory Reuse and Data Migration for Extreme-Scale Deep Learning on Many-Core Architectures. *ACM Transactions on Architecture and Code Optimization* (2018).
- [31] Sang-Woo Jun, Ming Liu, Sungjin Lee, Jamey Hicks, John Ankcorn, Myron King, Shuotao Xu, and Arvind. 2015. BlueDBM: An Appliance for Big Data Analytics. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [32] Sang-Woo Jun, Andy Wright, Sizhuo Zhang, Shuotao Xu, and Arvind. 2018. GraBoost: Using Accelerated Flash Storage for External Graph Analytics. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [33] Yangwook Kang, Yang-suk Kee, Ethan L Miller, and Chanik Park. 2013. Enabling Cost-Effective Data Processing With Smart SSD. In *Proceedings of the IEEE Symposium on Mass Storage Systems and Technologies (MSST)*.
- [34] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S. Lee, Meng Li, Bert Maher, Dheevatsa Mudigere, Maxim Naumov, Martin Schatz, Mikhail Smelyanskiy, Xiaodong Wang, Brandon Reagen, Carole-Jean Wu, Mark Hempstead, and Xuan Zhang. 2020. RecNMP: Accelerating Personalized Recommendation with Near-Memory Processing. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [35] Kimberly Keeton, David A Patterson, and Joseph M Hellerstein. 1998. A Case for Intelligent Disks (IDISks). *Acm Sigmod Record* (1998).
- [36] Byeongho Kim, Jaehyun Park, Eojin Lee, Minsoo Rhu, and Jung Ho Ahn. 2020. TRIM: Tensor Reduction in Memory. In *IEEE Computer Architecture Letters*.
- [37] Duckhwan Kim, Jaeha Kung, Sek Chai, Sudhakar Yalamanchili, and Saibal Mukhopadhyay. 2016. Neurocube: A Programmable Digital Neuromorphic Architecture with High-Density 3D Memory. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [38] Minsub Kim and Sungjin Lee. 2020. Reducing Tail Latency of DNN-based Recommender Systems using In-Storage Processing. In *Proceedings of the ACM SIGOPS Asia-Pacific Workshop on Systems (APSys)*.
- [39] Sungchan Kim, Hyunok Oh, Chanik Park, Sangyeun Cho, Sang-Won Lee, and Bongki Moon. 2016. In-Storage Processing of Database Scans and Joins. *Information Sciences* (2016).
- [40] Kevin Kinningham, Christopher Re, and Philip Levis. 2020. GRIP: A Graph Neural Network Accelerator Architecture. *arXiv preprint arXiv:2007.13828* (2020).
- [41] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [42] Gunjae Koo, Kiran Kumar Matam, Te I., H.V. Krishna Giri Narra, Jing Li, Hung-Wei Tseng, Steven Swanson, and Murali Annavaram. 2017. Summarizer: Trading Communication with Computing Near Storage. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [43] Jaewook Kwak, Sangjin Lee, Kibin Park, Jinwoo Jeong, and Yong Ho Song. 2020. Cosmos+ OpenSSD: Rapid Prototype for Flash Storage Systems. *ACM Transactions on Storage* (2020).
- [44] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. 2019. TensorDIMM: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [45] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. 2021. Tensor Casting: Co-Designing Algorithm-Architecture for Personalized Recommendation Training. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*.
- [46] Youngeun Kwon and Minsoo Rhu. 2018. A Case for Memory-Centric HPC System Architecture for Training Deep Neural Networks. In *IEEE Computer Architecture Letters*.
- [47] Youngeun Kwon and Minsoo Rhu. 2018. Beyond the Memory Wall: A Case for Memory-Centric HPC System for Deep Learning. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [48] Youngeun Kwon and Minsoo Rhu. 2019. A Disaggregated Memory System for Deep Learning. In *IEEE Micro*.
- [49] Jinho Lee, Heesu Kim, Sungjoo Yoo, Kiyoung Choi, H Peter Hofstee, Gi-Joon Nam, Mark R Nutter, and Damir Jasek. 2017. Extrav: Boosting Graph Processing Near Storage with a Coherent Accelerator. *Proceedings of the VLDB Endowment (PVLDB)* (2017).
- [50] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyoungwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [51] Yunjae Lee, Youngeun Kwon, and Minsoo Rhu. 2021. Understanding the Implication of Non-Volatile Memory for Large-Scale Graph Neural Network Training. In *IEEE Computer Architecture Letters*.
- [52] Young-Sik Lee, Luis Cavazos Quero, Sang-Hoon Kim, Jin-Soo Kim, and Seungryoul Maeng. 2016. ActiveSort: Efficient External Sorting Using Active SSDs in the MapReduce Framework. *Future Generation Computer Systems* (2016).
- [53] J. Leskovec, J. Kleinberg, and C. Faloutsos. 2005. Graphs over Time: Densification Laws, Shrinking Diameters and Possible Explanations. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.
- [54] Canguan Li, Ying Wang, Cheng Liu, Shengwen Liang, Huawei Li, and Xiaowei Li. 2021. GLIST: Towards In-Storage Graph Learning. In *Proceedings of USENIX Conference on Annual Technical Conference (ATC)*.
- [55] Jiajun Li, Ahmed Louri, Avinash Karanth, and Razvan Bunescu. 2021. GCNAX: A Flexible and Energy-Efficient Accelerator for Graph Convolutional Neural Networks. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*.

- [56] Shengwen Liang, Ying Wang, Cheng Liu, Lei He, Huawei Li, Dawen Xu, and Xiao-Wei Li. 2020. EnGN: A High-Throughput and Energy-Efficient Accelerator for Large Graph Neural Networks. *IEEE Trans. Comput.* (2020).
- [57] Zhiqi Lin, Cheng Li, Youshan Miao, Yunxin Liu, and Yulong Xu. 2020. PaGraph: Scaling GNN Training on Large Graphs via Computation-Aware Caching. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*.
- [58] Kiran Kumar Matam, Gunjae Koo, Haipeng Zha, Hung-Wei Tseng, and Murali Annavaram. 2019. GraphSSD: Graph Semantics Aware SSD. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [59] Jason Mohoney, Roger Waleffe, Henry Xu, Theodoros Rekatsinas, and Shivaram Venkataraman. 2021. Marius: Learning Massive Graph Embeddings on a Single Machine. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [60] Jaehyun Park, Byeongho Kim, Sungmin Yun, Eojin Lee, Minsoo Rhu, and Jung Ho Ahn. 2021. TRiM: Enhancing Processor-Memory Interfaces with Scalable Tensor Reduction in Memory. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [61] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based GPU Memory Management for Deep Learning. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [62] Jie Ren, Jiaolin Luo, Kai Wu, Minjia Zhang, Hyeran Jeon, and Dong Li. 2021. Sentinel: Efficient Tensor Migration and Allocation on Heterogeneous Memory Systems for Deep Learning. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*.
- [63] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. 2016. vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [64] Minsoo Rhu, Mike O'Connor, Niladrish Chatterjee, Jeff Pool, Youngeun Kwon, and Stephen W. Keckler. 2018. Compressing DMA Engine: Leveraging Activation Sparsity for Training Deep Neural Networks. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*.
- [65] Erik Riedel, Garth Gibson, and Christos Faloutsos. 1998. Active Storage for Large-Scale Data Mining and Multimedia Applications. In *Proceedings of Conference on Very Large Databases (VLDB)*.
- [66] Samsung. 2021. SmartSSD. <https://www.xilinx.com/applications/data-center/computational-storage/smartssd.html>
- [67] Sudharsan Seshadri, Mark Gahagan, Sundaram Bhaskaran, Trevor Bunker, Arup De, Yanqin Jin, Yang Liu, and Steven Swanson. 2014. Willow: A User-Programmable SSD. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [68] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [69] Xinkai Song, Tian Zhi, Zhe Fan, Zhenxing Zhang, Xi Zeng, Wei Li, Xing Hu, Zidong Du, Qi Guo, and Yunji Chen. 2021. Cambricon-G: A Polyvalent Energy-Efficient Accelerator for Dynamic Graph Neural Networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2021).
- [70] Qidong Su, Minjie Wang, Da Zheng, and Zheng Zhang. 2021. Adaptive Load Balancing for Parallel GNN Training. In *Proceedings of MLSys Workshop on Graph Neural Networks and Systems (GNNs)*.
- [71] NGD Systems. 2021. Newport Platform. <https://www.ngdsystems.com/solutions#NewportSection>
- [72] Devesh Tiwari, Simona Boboila, Sudharshan Vazhkudai, Youngjae Kim, Xiaosong Ma, Peter Desnoyers, and Yan Solihin. 2013. Active Flash: Towards Energy-Efficient, In-Situ Data Analytics on Extreme-Scale Machines. In *Proceedings of USENIX Conference on File and Storage Technologies (FAST)*.
- [73] Mahdi Torabzadehkashi, Siavash Rezaei, Ali Heydarigorji, Hosein Bobarshad, Vladimir Alves, and Nader Bagherzadeh. 2019. Catalina: In-Storage Processing Acceleration for Scalable Big Data Analytics. In *Proceedings of the Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*.
- [74] Hung-Wei Tseng, Qianchen Zhao, Yuxiao Zhou, Mark Gahagan, and Steven Swanson. 2016. Morpheus: Creating Application Objects Efficiently for Heterogeneous Computing. In *Proceedings of the International Symposium on Computer Architecture (ISCA)*.
- [75] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph Attention Networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [76] Jianguo Wang, Dongchul Park, Yang-Suk Kee, Yannis Papakonstantinou, and Steven Swanson. 2016. SSD In-Storage Computing for List Intersection. In *Proceedings of the International Workshop on Data Management on New Hardware (DaMoN)*.
- [77] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU Memory Management for Training Deep Neural Networks. In *Proceedings of the Symposium on Principles and Practice of Parallel Programming (PPOPP)*.
- [78] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, and Zheng Zhang. 2019. Deep Graph Library: A Graph-Centric, Highly-Performant Package for Graph Neural Networks. *arXiv preprint arXiv:1909.01315* (2019).
- [79] Xiaowei Wang, Jiecao Yu, Charles Augustine, Ravi Iyer, and Reetuparna Das. 2019. Bit Prudent In-Cache Acceleration of Deep Convolutional Neural Networks. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*.
- [80] Yuke Wang, Boyuan Feng, Gushu Li, Shuangchen Li, Lei Deng, Yuan Xie, and Yufei Ding. 2021. GNNAdvisor: An Adaptive and Efficient Runtime System for GNN Acceleration on GPUs. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [81] Mark Wilkening, Udit Gupta, Samuel Hsia, Caroline Trippel, Carole-Jean Wu, David Brooks, and Gu-Yeon Wei. 2021. RecSSD: Near Data Processing for Solid State Drive Based Recommendation Inference. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [82] Louis Woods, Zolt István, and Gustavo Alonso. 2014. Ibox: An Intelligent Storage Engine with Support for Advanced SQL Offloading. *Proceedings of the VLDB Endowment (PVLDB)* (2014).
- [83] Shuotao Xu, Thomas Bourgeat, Tianhao Huang, Hojun Kim, Sungjin Lee, and Arvind Arvind. 2020. AQUOMAN: An Analytic-Query Offloading Machine. In *Proceedings of the International Symposium on Microarchitecture (MICRO)*.
- [84] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. 2020. HyGCN: A GCN Accelerator with Hybrid Architecture. In *Proceedings of the International Symposium on High-Performance Computer Architecture (HPCA)*.
- [85] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.
- [86] Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L. Hamilton, and Jure Leskovec. 2018. Hierarchical Graph Representation Learning with Differentiable Pooling. In *Proceedings of the International Conference on Neural Information Processing Systems (NIPS)*.
- [87] Hanqing Zeng and Viktor Prasanna. 2020. GraphACT: Accelerating GCN Training on CPU-FPGA Heterogeneous Platforms. In *Proceedings of the ACM International Symposium on Field-Programmable Gate Arrays (FPGA)*.
- [88] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2020. GraphSAINT: Graph Sampling Based Inductive Learning Method. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [89] Da Zheng, Chao Ma, Minjie Wang, Jinjing Zhou, Qidong Su, Xiang Song, Quan Gan, Zheng Zhang, and George Karypis. 2021. DistDGL: Distributed Graph Neural Network Training for Billion-Scale Graphs. *arXiv preprint arXiv:2010.05337* (2021).
- [90] Rong Zhu, Kun Zhao, Hongxia Yang, Wei Lin, Chang Zhou, Baole Ai, Yong Li, and Jingren Zhou. 2019. AliGraph: A Comprehensive Graph Neural Network Platform. *Proceedings of the VLDB Endowment (PVLDB)* (2019).